



UNIVERSIDAD TECNICA
FEDERICO SANTA MARIA



DEPARTAMENTO DE
ELECTRONICA

Documentación Tarea 3

Simulador Gráfico en C++ y Qt de EloTelTags y Aplicación Find My

Fecha 30 de junio de 2026

Integrantes: Rodrigo Hernández
Ignacio Torres

Asignatura: Programación Orientada a Objetos

Profesor: Agustin Gonzalez

Índice

1	Introducción	3
1.1	Objetivos	3
2	Desarrollo	4
2.1	Interacción entre clases durante la ejecución	4
2.2	Diagrama de Clases	4
2.3	Dificultades y Soluciones	5
3	Conclusiones	6
4	Bibliografía	7

Índice de figuras

1. Introducción

1.1. Objetivos

- 1.
- 2.
- 3.
- 4.

2. Desarrollo

2.1. Interacción entre clases durante la ejecución

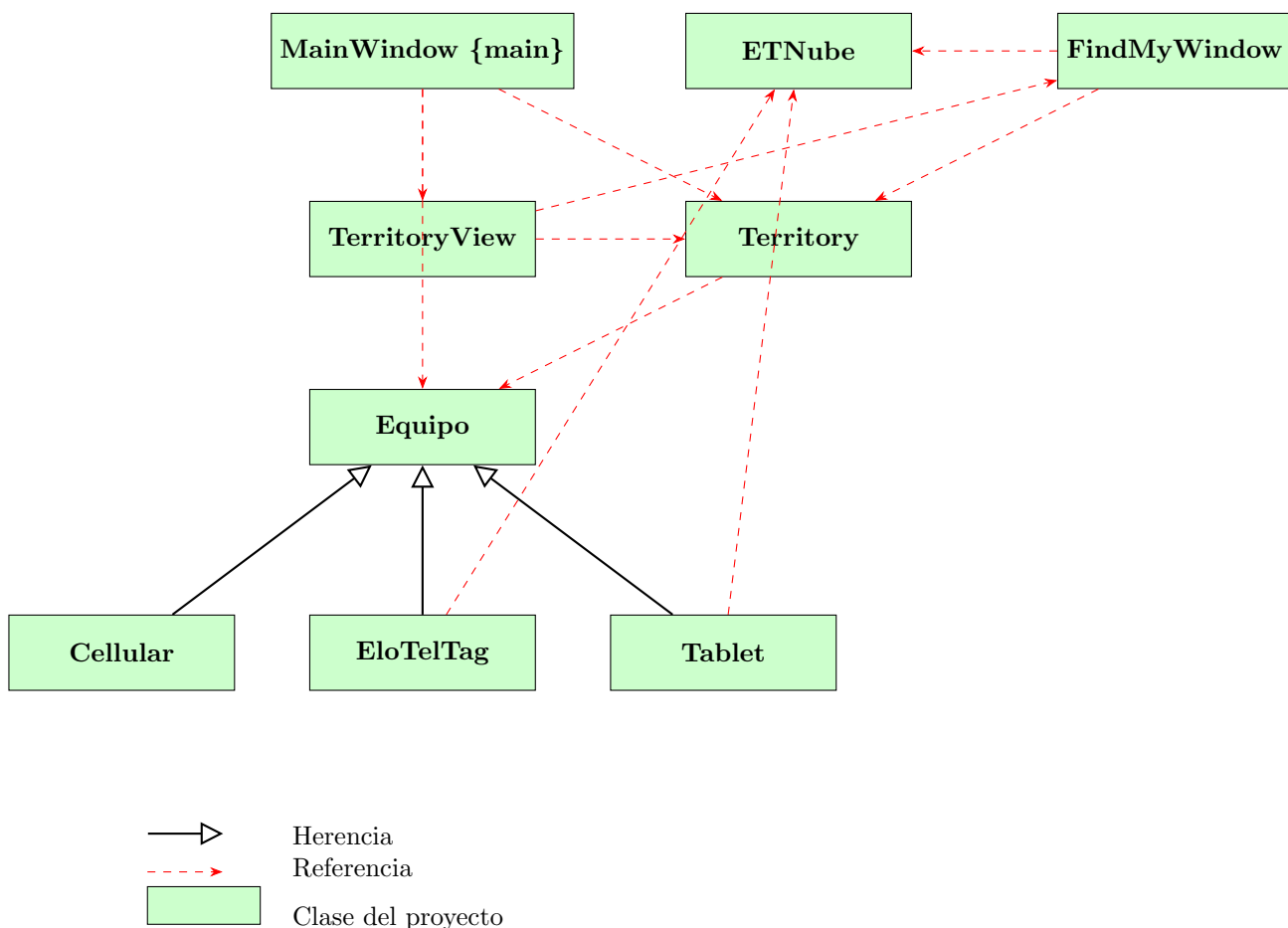
Cuando se ejecuta el programa, `main.cpp` es lo primero que arranca, creando la `QApplication` y la instancia de `MainWindow`. Esta última le pide al usuario que elija el archivo de configuración mediante un `QFileDialog`, y con un `ifstream` va leyendo la imagen de fondo, el paso de tiempo y los datos de cada persona. Con esa información crea los objetos `Territory` y `TerritoryView`, y por cada persona arma su `Cellular`, sus `EloTelTag` y su `Tablet` si es que tiene, agregándolos todos al `Territory`.

Una vez que el usuario selecciona Play en el menú Simulation, se activa un `QTimer` que cada `timeStep` segundos llama al slot `advance()` en `MainWindow`. Ahí es donde pasa todo: primero se recorre el vector de equipos de `Territory` llamando a `move()` sobre cada uno, lo que actualiza sus coordenadas y hace que reboten al llegar a un borde del territorio. Luego se llama a `update()` sobre cada equipo, método virtual que en `EloTelTag` y `Tablet` revisa sus contadores internos de tiempo: cada 4 segundos los tags y cada 5 segundos las tablets activan su propiedad `bipping`, que dispara la animación de un radar (círculo que crece de 0 a 50 píxeles en 1 segundo). Como en C++ y Qt no existen los *bindings* automáticos de JavaFX, al final de `advance()` se invoca manualmente `territoryView->update()`, lo que provoca que `paintEvent()` repinte la imagen de fondo y la posición actual de cada equipo según el tipo retornado por `getType()`.

Si se hace click en un celular o tablet, se detecta el equipo bajo el cursor y despliega un emergente con la opción “Find My”. Al seleccionarla se instancia una `FindMyWindow`, la cual consulta en `EtNube` y muestra dónde están los bienes de esa persona. Un `QTimer` interno refresca esa ventana cada 1 segundo para que las posiciones se vayan actualizando. Cada equipo listado incluye además un botón “Traza” que invoca sobre el equipo correspondiente, activando o desactivando el dibujo de su trayectoria en `TerritoryView`.

2.2. Diagrama de Clases

A continuación se presenta el diagrama de clases de la aplicación en su etapa final (Etapa 4), que muestra las relaciones de herencia y referencia entre las clases del proyecto.



El diagrama muestra cómo está organizado el simulador siguiendo una separación Modelo-Vista similar a un patrón MVC.

La clase **Equipo** es la base de la jerarquía de modelos. De ahí heredan **Cellular**, **EloTelTag** y **Tablet**, cada uno implementando `getType()` y, en el caso de los dos últimos, su propia lógica de `update()` para el ciclo de señal de búsqueda y registro de traza.

Territory actúa como controlador del modelo: mantiene el vector de equipos, los mueve y delega en cada uno la actualización de su estado interno, sin necesidad de conocer el tipo concreto de cada equipo gracias al polimorfismo de **Equipo**.

TerritoryView es la vista principal: dibuja la imagen de fondo y, sobre ella, cada equipo según su tipo (color y forma distintos), el radar de búsqueda y las trazas activas. A diferencia de la versión en JavaFX, no existe una vista independiente por cada tipo de equipo; todo el dibujo se centraliza en el método `paintEvent()` de **TerritoryView**, distinguiendo cada caso mediante `getType()`.

ETNube es donde se guardan todas las posiciones reportadas, usando un `std::map` que asocia el dueño y el dispositivo con su último reporte. Tanto **TerritoryView** (al detectar el click) como **FindMyWindow** (al refrescar su contenido) interactúan con esta clase.

FindMyWindow consulta a **ETNube** y muestra la información de los bienes de la persona seleccionada, incluyendo el botón de traza para cada uno.

Y **MainWindow** es la clase principal: lee el archivo de configuración, crea los objetos del modelo, configura el **QTimer** de la simulación y levanta la interfaz que se observa al ejecutar el programa.

2.3. Dificultades y Soluciones

- 1. La imagen de fondo no se cargaba al ejecutar desde Qt Creator.** Al ejecutar **Stage1** por primera vez, la ventana se mostraba completamente vacía: ni la imagen **Placeres.jpg** ni los equipos aparecían en pantalla. El código de **TerritoryView** cargaba la ruta tal como venía en **config.txt**, pero Qt Creator ejecuta el binario desde su propio directorio de *build*, no desde la carpeta donde están los archivos fuente y los recursos del proyecto. Como `QPixmap::load()` no encontraba el archivo en esa ruta, **backgroundImage** quedaba nula y, al llamar `setFixedSize(backgroundImage.size())`, la vista tomaba tamaño 0x0, dejando la ventana en blanco sin ningún mensaje de error visible. La solución fue: se configuró el directorio de trabajo (*Working directory*) en la pestaña *Projects* → *Run* de Qt Creator apuntando a la carpeta donde residen **Placeres.jpg** y **config.txt**.
- 2. Los equipos no se movían al seleccionar Play en la simulación.** Durante el desarrollo de la Etapa 2 se agregó un **QTimer** en **MainWindow**, para mover periódicamente todos los equipos. Sin embargo, al ejecutar el programa y seleccionar la opción “Play” del menú *Simulation*, los equipos permanecían estáticos en su posición inicial. El problema era que el **QTimer** se creaba y se conectaba a su *slot* en el constructor de **MainWindow**, pero nunca se invocaba `timer->start()` en ese punto: dicha llamada solo existía dentro de `play()`. Al revisar la conexión entre el menú se confirmó que la señal sí se emitía correctamente, por lo que se agregó una impresión temporal para verificar si el *slot* se estaba llamando, constatando que el temporizador nunca llegaba a iniciar su ciclo. La solución fue invocar `timer->start(timeStep * 1000)` directamente en el constructor de **MainWindow**, de modo que la simulación comience a moverse automáticamente al cargar la configuración, dejando los `play()` y `pause()` encargados únicamente de reanudar o detener el temporizador ya inicializado.

3. Conclusiones

4. Bibliografía